

A COOKBOOK FOR NODE.JS DEVELOPERS

n8n Automation Cookbook

Build, secure, and self-host real workflow automation — for a developer who already knows how to write the glue code by hand.

n8n

Workflows

Webhooks

Code node

Expressions

AI Agents

Queue mode

Self-hosting

Written for engineers fluent in Node.js / TypeScript who want automations that ship — visual where it helps, code where it counts.

Every pattern answers: what it solves, how you'd do it by hand vs in n8n, the failure modes, and when NOT to use n8n at all.

Edition 2026 · nodes · items · reliability · self-hosting

Table of Contents

How to Use This Book	4
The one idea: a workflow is a pipeline of nodes passing items	4
What this book covers	5
Part 1 • What n8n Is (and When to Reach for It)	6
1.1 Where it sits, mapped to what you know	6
1.2 What makes it developer-first	6
1.3 The good use cases	7
1.4 When NOT to use n8n	7
Part 2 • Running n8n (Docker) & the Editor	8
2.1 The one-liner	8
2.2 A persistent-enough dev setup (compose)	8
2.3 A tour of the editor	9
2.4 The Community vs Cloud vs Enterprise split	9
Part 3 • Nodes, Triggers & Executions	11
3.1 Triggers — what starts a workflow	11
3.2 Nodes — the steps	11
3.3 Branching and looping	12
3.4 Executions — your logbook and debugger	12
Part 4 • The Data Model: Items & Expressions	13
4.1 Everything is a list of items	13
4.2 Expressions — the <code>{{ }}</code> syntax	13
4.3 Reshaping data without code	14
Part 5 • Talking to APIs: the HTTP Request Node	16
5.1 The node is fetch, with a UI	16
5.2 Building requests from items	16
5.3 Pagination, timeouts, and batching	17
5.4 When there is a dedicated node	17
Part 6 • The Code Node (Real JavaScript)	18
6.1 Two modes: per-item vs all-items	18
6.2 What you can and can't do	19
6.3 Python in the Code node	19
6.4 The escape hatch pattern	19
Part 7 • Credentials & Secrets	21
7.1 Credentials are separate from workflows	21
7.2 The encryption key is everything	21
7.3 Auth types you'll use	22
7.4 External secret stores (Enterprise)	22
Part 8 • Error Handling & Reliability	23
8.1 Per-node retries and timeouts	23
8.2 Continue-on-error and error branches	23

8.3 The global Error Trigger workflow	23
8.4 Idempotency — the reliability chapter within the chapter.	24
Part 9 • AI Workflows: LLMs, Agents & RAG.	25
9.1 The building blocks	25
9.2 A basic LLM chain	25
9.3 An agent with a tool	26
9.4 Retrieval (RAG) as a workflow	26
9.5 MCP in n8n — tools over a standard protocol	26
Part 10 • Self-Hosting: Persistence, Queue Mode & Security	28
10.1 Move the database to Postgres.	28
10.2 Queue mode — the part that makes it scale	29
10.3 Security essentials	29
Part 11 • Production Practices	31
11.1 Version control your workflows.	31
11.2 Environments: dev vs prod.	31
11.3 Testing	31
11.4 Monitoring	32
11.5 The honest "when NOT to use n8n" list	32
Part 12 • Capstone & Cheat Sheet	33
12.1 Capstone — build one real automation end to end.	33
12.2 Node.js → n8n mental-model map	33
12.3 Expressions cheat sheet	34
12.4 Self-hosting env cheat sheet.	34
12.5 The five rules.	34

How to Use This Book

You already know how to automate things: you write an Express route for the webhook, a cron job for the schedule, some `fetch` calls to stitch APIs together, a BullMQ worker for the slow parts, and a `try/catch` with a Slack alert when it breaks. It works — and it's also a pile of small services you have to deploy, monitor, and maintain forever, most of which do nothing but move JSON from one place to another.

n8n is a tool for building exactly that kind of glue *visually* — a workflow of connected nodes on a canvas — while still letting you drop into real JavaScript whenever the visual nodes aren't enough. It's built on Node.js, it's open-source, and you self-host it. This book teaches it to someone who could already write the code by hand, so every chapter is honest about the trade: what n8n saves you, where it gets in the way, and when you should just write the service instead.

Each chapter answers four questions:

1. **What problem does this solve?**
2. **How would I do it by hand (Node) vs in n8n?**
3. **What are the failure modes?**
4. **When should I *not* use n8n for this?**

The one idea: a workflow is a pipeline of nodes passing items

Every n8n workflow is the same shape. A **trigger node** starts an execution (a webhook arrives, a timer fires, a manual click). Each node after it **receives a list of items** (JSON objects), does one thing — call an API, transform data, run code, branch — and **passes items to the next node**. That's it: data flows left to right through nodes.

DIAGRAM

```
Trigger → Node → Node → Node → (done)
(webhook) (HTTP) (transform) (Slack)
every node gets [ {json:{...}}, {json:{...}} ] and returns the same shape
```

If you think of it as a Unix pipe (`curl | jq | ...`) where each stage is a node and the data between stages is always a list of JSON items, you have the whole mental model.

NODE.JS MENTAL MODEL

A workflow is the request handler you'd write, decomposed into steps: the trigger is your route or cron registration, each node is a function in the chain, and the "items" flowing between nodes are the array you'd `.map()` over. The Code node is the point where you stop configuring and just write the function yourself.

What this book covers

Setup and the editor, the node/trigger/execution model, the data model (items and expressions), calling APIs, the Code node, credentials and secrets, error handling, AI workflows (LLMs, agents, and retrieval), and — the part that decides whether n8n is a toy or infrastructure — **self-hosting it for real**: Postgres persistence, queue mode with workers, and security. It ends with a capstone and a cheat sheet.

Throughout, it assumes you have the companion **Background Jobs & Queues** cookbook nearby, because the honest comparison to writing a BullMQ worker is the fastest way to understand what n8n is and isn't for.

HOW TO READ IT

Read Parts 1-4 in order — they build the mental model. After that, treat it as a reference: jump to the Code node, error handling, AI, or self-hosting chapter when you need it. Do the exercises with n8n actually running (Part 2) — this is a tool you learn by clicking, not by reading.

Part 1 · What n8n Is (and When to Reach for It)

n8n (pronounced "n-eight-n," short for *nodemation*) is a **source-available workflow-automation tool**. You build automations by wiring nodes together on a canvas; it runs on Node.js; and you can self-host it. Think Zapier or Make, but developer-first: open, self-hostable, and with a real code escape hatch.

1.1 Where it sits, mapped to what you know

You want to...	By hand (Node)	Managed SaaS	n8n
React to an event	Express webhook route	Zapier trigger	Webhook node
Run on a schedule	cron / <code>node-cron</code>	Zapier schedule	Schedule trigger
Call several APIs in order	<code>fetch</code> + <code>await</code>	Zap steps	HTTP Request nodes
Transform data	JS in your handler	limited field mapping	Set / Code node
Do slow work reliably	BullMQ worker	(hidden)	queue mode + workers
Own your data & secrets	yes	no	yes (self-hosted)
Change it without a deploy	no (redploy)	yes	yes (edit canvas)

The two columns that matter to you are the ends. Versus **writing it by hand**, n8n trades some control and raw performance for enormous speed-of-change and a library of ~400 pre-built integrations. Versus **Zapier/Make**, it trades a polished SaaS for self-hosting, data ownership, and a Code node where you can do anything Node can.

1.2 What makes it developer-first

- **It's Node under the hood.** The whole thing is TypeScript/Node. The Code node runs real JavaScript (or Python via Pyodide), so you're never boxed in.
- **Self-hosted.** One Docker command and it's yours — your data, your credentials, your network. No per-task pricing, no third party holding your keys.
- **A generic HTTP Request node.** Even when there's no dedicated integration for a service, if it has a REST API, n8n can call it.
- **Fair-code / source-available.** You can read the source, self-host freely, and extend it with custom nodes. (It is *source-available*, not classic OSI open-source — the license restricts reselling it as a competing hosted service.)

NODE.JS MENTAL MODEL

n8n is a Node app that executes a graph you draw. Each node is essentially a typed function `(items) => items`; the canvas is the composition. When you hit a Code node, you're literally writing that function body in JavaScript. Nothing magic is happening — it's the orchestration and the 400 pre-written functions you're getting for free.

1.3 The good use cases

n8n shines at **glue and integration** work that changes often:

- Internal automations: sync data between systems, notify on events, enrich records, back up data on a schedule.
- Lightweight backend glue: webhook → transform → store/notify, without standing up a service for each one.
- Ops/notifications: watch something, alert a channel, open a ticket.
- AI workflows: chain LLM calls, build a small agent, or a retrieval pipeline (Part 9) — visually, with vector-store and model nodes built in.
- Prototyping: wire an integration in minutes to see if it's worth building properly.

1.4 When NOT to use n8n

Be honest about the limits — this is the most important section in the chapter.

WHEN NOT TO USE N8N

Reach for code instead when the work is: **high-throughput or latency-critical** (n8n adds per-node overhead; a tight BullMQ worker will outrun it); **logic-heavy** (deeply nested branching and complex state is clearer in a real codebase than in a sprawling canvas); **a core product feature** users depend on with strict SLAs and test coverage; or **tiny and stable** (a five-line script on a cron doesn't need a workflow engine). n8n earns its keep on integration glue that changes often and benefits from being visible and editable — not on your hot path.

COMMON MISTAKE

Treating n8n as a general-purpose application platform. People build a 50-node monster with intricate branching to avoid writing code, then can't debug or version it. If a workflow has grown into a program, extract the hard logic into a Code node — or a real service the workflow calls — and keep the canvas as the orchestration layer, not the whole app.

EXERCISE 1.1

List three automations you currently run (or would run) by hand — a webhook handler, a scheduled sync, an alerting script. For each, note what writing it by hand costs you (a deploy, a service to monitor) and whether n8n's trade — less control, more speed-of-change — is worth it. Keep the list; you'll build one of them as the capstone (Part 12).

Part 2 · Running n8n (Docker) & the Editor

You self-host n8n, and the fastest way is Docker — the same way you run your learning plans. This chapter gets it running and oriented; Part 10 hardens it for real use.

2.1 The one-liner

SHELL

```
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

Open **http://localhost:5678**, create the owner account, and you're in. The `-v n8n_data:/home/node/.n8n` volume matters: without it, every workflow and credential lives inside the container and vanishes when it's removed. (That named volume is the *minimum*; Part 10 moves the real data into Postgres.)

COMMON MISTAKE

Running the container with no volume, building workflows for an hour, then `docker rm`-ing it and losing everything. The default database is a SQLite file *inside* the container. Always mount `/home/node/.n8n` — or, better, use the Postgres setup from Part 10 from the start if this is more than a spike.

2.2 A persistent-enough dev setup (compose)

For day-to-day learning, a tiny compose file with a volume and a couple of env vars is nicer than the one-liner:

YAML

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    ports:
      - "5678:5678"
    environment:
      - GENERIC_TIMEZONE=Europe/Amsterdam      # correct times for Schedule
triggers
      - N8N_SECURE_COOKIE=false                # dev only: allow http on localhost
    volumes:
```

```
- n8n_data:/home/node/.n8n
volumes:
  n8n_data:
```

SHELL

```
docker compose up -d          # http://localhost:5678
```

NODE.JS MENTAL MODEL

This is your `docker compose up` for any Node service — image, port, env, a volume for state. The only n8n-specific idea is *what* the state is: workflows, credentials, and execution history, all in the mounted data dir (or Postgres).

2.3 A tour of the editor

Five things to find, because the rest of the book refers to them:

- **Canvas** — where you place and connect nodes. Drag from a node's output dot to another node's input to wire them.
- **The node panel** (`Tab` / the `+`) — search 400+ nodes. Every workflow starts with a **trigger** node.
- **Execute** — run the workflow (or a single node) on demand. Each node shows its output data inline after running — this is your `console.log`.
- **Executions** — the history of past runs (manual and automatic), each with the exact data at every node. This is your logbook and debugger.
- **Pin data** — freeze a node's output to a sample so you can iterate on downstream nodes without re-hitting an API upstream. Enormously useful.

2.4 The Community vs Cloud vs Enterprise split

You'll hit feature gates, so know the map: **self-hosted Community** (free, what this book uses) covers everything you need to learn and run real automations. **n8n Cloud** is their hosted version (you skip ops). **Enterprise** adds team/governance features (SSO, environments, advanced RBAC, external secret stores). Nothing in this book requires Cloud or Enterprise.

PRODUCTION TIP

Set `GENERIC_TIMEZONE` (and the container's `TZ`) explicitly. Schedule triggers fire in the instance timezone, and the classic self-host bug is a cron that runs an hour off — or on the wrong day — because the container defaulted to UTC. Set it once and your schedules mean what you think.

EXERCISE 2.1

Get n8n running with the compose file above, create the owner account, and confirm the data survives a restart (`docker compose down && up` , workflows still there). Then build the tiniest possible workflow — a Manual Trigger into an Edit Fields (Set) node that outputs `{ "hello": "n8n" }` — execute it, and find that run in the Executions list.

Part 3 · Nodes, Triggers & Executions

Three concepts run the whole tool: **nodes** (the steps), **triggers** (what starts a run), and **executions** (a run and its recorded data). Get these right and everything else is detail.

3.1 Triggers — what starts a workflow

A workflow does nothing until a trigger fires. The ones you'll use constantly:

Trigger	Fires when...	By-hand equivalent
Manual	you click Execute	running a script yourself
Schedule	a timer/cron matches	<code>node-cron</code> / a cron entry
Webhook	an HTTP request hits its URL	an Express route
App triggers	an event in a connected app (new row, new email, new message)	that app's webhook/polling

SHELL

```
# a Webhook trigger gives you a URL; hitting it starts the workflow with the body as items
curl -X POST https://your-n8n/webhook-test/abc123 \
  -H 'content-type: application/json' \
  -d '{"user_id": 42, "action": "signup"}'
```

NODE.JS MENTAL MODEL

A Webhook trigger is `app.post("/hook", handler)` — except n8n gives you two URLs: a **test** URL (active only while you're editing, so you can click Execute and watch the data) and a **production** URL (live once the workflow is activated). Forgetting to *activate* the workflow — leaving it on the test URL — is the #1 "my webhook doesn't fire" cause.

3.2 Nodes — the steps

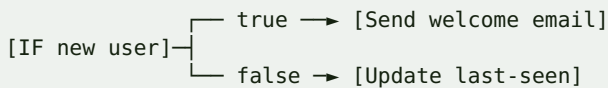
After the trigger, each node does one job. The families you'll meet:

- **Action nodes** — do something in a service (send a Slack message, insert a Postgres row, create a Notion page).
- **Transform nodes** — reshape data: **Edit Fields (Set)**, **Filter**, **Split Out**, **Aggregate**, **Merge**.
- **Flow nodes** — control the path: **IF**, **Switch**, **Loop Over Items**, **Merge**, **Stop And Error**.

- **Core nodes** — **HTTP Request** (call anything), **Code** (write JS), **Execute Sub-workflow** (call another workflow like a function).

3.3 Branching and looping

DIAGRAM



IF splits into two paths; **Switch** into many (a `switch` statement). Most nodes **loop automatically** — if 50 items arrive, an HTTP Request node runs 50 times, once per item — so you rarely need an explicit loop. When you do (e.g. to batch, or to call something that must be sequential), **Loop Over Items** batches them.

COMMON MISTAKE

Adding a manual loop out of Node reflex. In n8n, passing 100 items into an HTTP Request node already makes 100 calls — wrapping it in a Loop Over Items *and* expecting per-item behavior double-processes or confuses the data flow. Assume nodes iterate over items for you; reach for an explicit loop only for batching or forced-sequential work.

3.4 Executions — your logbook and debugger

Every run is recorded as an **execution**: which nodes ran, the exact items in and out of each, and any error. This is the single best thing about debugging in n8n — you don't add `console.log`, you open the execution and read the data at each step.

- **Manual executions** happen when you click Execute (great for building).
- **Production executions** happen when a live trigger fires.
- You control retention (how long history is kept) via env vars (Part 10) — it can grow fast on a busy instance.

PRODUCTION TIP

Turn on "**Save execution data**" for failed runs at minimum, and prune successful ones aggressively (`EXECUTIONS_DATA_MAX_AGE` , `..._PRUNE`). Full execution data on every success is the fastest way to bloat your database on a high-volume instance — keep failures for debugging, discard the noise.

EXERCISE 3.1

Build three trigger workflows: (1) a Schedule trigger that runs every minute and outputs the current time; (2) a Webhook trigger you POST to with `curl`, printing the body; (3) an IF node that routes an incoming `{ "amount": n }` into "big" (≥ 100) or "small" paths. Run each, then open the Executions list and read the data at every node — no logging added.

Part 4 · The Data Model: Items & Expressions

This is the chapter that makes n8n click. Almost every confusion — "why is my node running twice?", "how do I reference that field?" — comes from not understanding **items**. Read this one slowly.

4.1 Everything is a list of items

Data between nodes is always an **array of items**, and each item is an object with a `json` property (and optionally `binary`):

JSON

```
[
  { "json": { "id": 1, "name": "Ada" } },
  { "json": { "id": 2, "name": "Grace" } }
]
```

A node **runs once per item**. If two items arrive at an HTTP Request node, it makes two requests. This is the whole reason you rarely write loops: the list is the loop.

NODE.JS MENTAL MODEL

Think of the data as the array you'd hold in a variable, and each node as a `.map()` (or `.filter()`, or an async call per element) over it. `item.json` is your object; the node body is the callback. "Runs once per item" is just "the framework maps your node over the array for you."

4.2 Expressions — the `{{ }}` syntax

Any field on any node can be a **fixed value** or an **expression**. Switch a field to expression mode and you write JavaScript inside `{{ }}`:

DIAGRAM

<code>{{ \$json.name }}</code>	→ the current item's name
<code>{{ \$json.email.toLowerCase() }}</code>	→ real JS methods work
<code>{{ \$json.total * 1.2 }}</code>	→ compute
<code>{{ \$now.toISO() }}</code>	→ current time (Luxon DateTime)
<code>{{ \$json.items.length }}</code>	→ array length

The important variables:

Expression	Means
<code>\$json</code>	the current item's <code>json</code> object
<code>\$json.field</code>	a field on it
<code>\$node["HTTP Request"].json</code>	output of a <i>named</i> earlier node
<code>\$items()</code>	all items (not just the current one)
<code>\$now</code> , <code>\$today</code>	Luxon date objects
<code>\$env.F00</code>	an environment variable (if allowed)
<code>\$vars</code> , <code>\$secrets</code>	instance variables / external secrets (Enterprise)

DIAGRAM

```
# reference an earlier node's data by name – the n8n version of using a prior result
{{ $node["Get User"].json.email }}
```

COMMON MISTAKE

Referencing `$json` and getting `undefined` because you meant a *different* node's output. `$json` is only the item from the **immediately preceding** node. To reach back to an earlier node (say the trigger's body after several transforms), name that node and use `$node["Name"].json` — or the current `$( 'Node Name' ).item.json` form. Renaming a node breaks these references, so fix them when you rename.

4.3 Reshaping data without code

Most transformations don't need the Code node:

- **Edit Fields (Set)** — build a clean output object; set fields with expressions.
- **Filter** — drop items that don't match a condition (like `.filter()`).
- **Split Out** — turn one item containing an array into many items (fan-out).
- **Aggregate** — combine many items into one (fan-in / `.reduce()`).
- **Merge** — join two branches (by position, by key, or append) — like `Promise.all` results combined, or a SQL join.

DIAGRAM

```
[1 item: {orders:[a,b,c]}] —Split Out→ [3 items: a, b, c]
[3 items: a, b, c] —Aggregate→ [1 item: {all:[a,b,c]}]
```

PRODUCTION TIP

Prefer Set / Filter / Split Out / Aggregate over a Code node for shape changes. They're readable at a glance on the canvas, they show their output inline, and a teammate (or you in six months) can see what's happening without reading JavaScript. Save the Code node for logic the built-in nodes genuinely can't express (Part 6).

EXERCISE 4.1

Start from one item containing `{ "users": [ {name,email}, {name,email} ] }`. Use **Split Out** to fan it into one item per user, a **Set** node with an expression to add `{{ $json.email.toLowerCase() }}` as `emailLower`, a **Filter** to keep only gmail addresses, and an **Aggregate** to collect the survivors back into a single list. Watch the item count change at each step in the node output.

Part 5 · Talking to APIs: the HTTP Request Node

Most real automations are "call this API, do something with the result, call that API." n8n has ~400 pre-built app nodes, but the one that makes it universal is the generic **HTTP Request** node — it can call anything with a REST API, which means you're never blocked waiting for an integration to exist.

5.1 The node is `fetch`, with a UI

TYPESCRIPT

```
// by hand
const res = await fetch("https://api.example.com/users?page=2", {
  method: "GET",
  headers: { Authorization: `Bearer ${token}` },
});
const data = await res.json();
```

The HTTP Request node is the same call, configured in fields: **Method**, **URL**, **Query Parameters**, **Headers**, **Body**, and **Authentication**. Its output is the parsed response as items — no `await res.json()` step, it's done for you.

NODE.JS MENTAL MODEL

It's `fetch` / `axios` with the options as form fields instead of an object literal. Query params, headers, and JSON body each get a little key/value editor, and any value can be an expression (`{{ $json.id }}`) so the request is built from the current item. Auth is the one thing you *don't* inline — that goes through Credentials (Part 7).

5.2 Building requests from items

Because a node runs once per item, the HTTP Request node is how you fan out calls:

DIAGRAM

```
[3 items: {id:1},{id:2},{id:3}]
|   URL: https://api.example.com/users/{{ $json.id }}
▼
[3 responses]      ← three GETs, one per item, automatically
```

Set the URL or body with expressions referencing `$json`, and n8n makes one request per item. This is your `Promise.all(ids.map(fetchUser))` — without writing the map.

5.3 Pagination, timeouts, and batching

The node has built-in help for the annoying parts:

- **Pagination** — a "Pagination" option follows `next` cursors / page params and returns all pages as items, so you don't hand-roll the loop.
- **Timeout & retries** — set a request timeout and on-fail retry count in the node's options (more on reliability in Part 8).
- **Batching / rate limits** — a "Batching" option throttles how many requests go out at once, so you don't hammer an API into a 429.

COMMON MISTAKE

Firing thousands of requests by passing a huge item list straight into the node with no batching, then getting rate-limited or IP-banned. When you're calling an external API per item, set the node's **Batch Size** and an interval — the visual equivalent of a `p-limit` concurrency cap. Respect the API's limits exactly as you would in code.

5.4 When there *is* a dedicated node

For popular services (Slack, Google Sheets, Postgres, Notion, GitHub...) a dedicated node is nicer than raw HTTP: it handles auth, exposes operations as dropdowns, and maps fields. Use it when it exists; fall back to HTTP Request for everything else, or when you need a call the dedicated node doesn't expose.

PRODUCTION TIP

For an API n8n doesn't have a node for, use the **"Import cURL"** button on the HTTP Request node: paste a working `curl` command from the API's docs and n8n fills in method, URL, headers, and body. It's the fastest way to go from "the docs have a curl example" to "a working node."

WHEN NOT TO REACH FOR THE HTTP NODE

If you're about to chain five HTTP Request nodes with heavy branching and data juggling between them to implement one coherent operation, consider writing that operation as a small service (or a single Code node calling the APIs) and letting the workflow call *it*. Five brittle HTTP nodes with expression glue are harder to test and change than one function.

EXERCISE 5.1

Call a public API (e.g. `https://api.github.com/users/{{ $json.user }}`) with the HTTP Request node, driven by a list of three usernames as items so it makes three calls. Add an Authentication credential (Part 7) with a token and confirm the rate limit rises. Then turn on pagination against an endpoint that returns pages and collect them all.

Part 6 · The Code Node (Real JavaScript)

This is the chapter that separates n8n from no-code tools and the reason a developer picks it. When the built-in nodes can't express what you need, the **Code node** lets you write real JavaScript (or Python) over the items. You're never stuck.

6.1 Two modes: per-item vs all-items

The Code node runs in one of two modes:

- **Run Once for All Items** — your code gets the whole `items` array; you return a new array. This is a `.map()` / `.reduce()` over everything.
- **Run Once for Each Item** — your code runs per item; you return one item. This is the body of a `.map()` callback.

JAVASCRIPT / NODE.JS

```
// Run Once for All Items – return an array of items
const items = $input.all(); // [{json:{...}}, ...]
return items.map(i => ({
  json: {
    id: i.json.id,
    fullName: `${i.json.first} ${i.json.last}`,
    createdAt: new Date().toISOString(),
  },
}));
```

JAVASCRIPT / NODE.JS

```
// Run Once for Each Item – return a single item
const data = $input.item.json;
return { json: { ...data, slug: data.title.toLowerCase().replace(/\s+/g, "-") } };
```

NODE.JS MENTAL MODEL

"All items" mode is `arr.map(...)` where you manage the array; "each item" mode is the callback body and n8n does the mapping. The one rule that trips people up: you must return **items** — objects shaped `{ json: { ... } }` (an array of them in all-items mode) — not a bare value. Returning `data` instead of `{ json: data }` is the classic first bug.

6.2 What you can and can't do

Inside the Code node you have normal JavaScript plus n8n helpers:

- `$input.all()` / `$input.item` — the incoming items.
- `$json`, `$node["Name"].json`, `$now` — the same expression variables.
- `$` helpers to reach other nodes' data.
- Standard JS: `fetch` (via helpers), `JSON`, `Date`, string/array methods, etc.

What you *can't* freely do: arbitrary `npm install`. The Code node runs in a sandboxed environment; external modules are restricted by default (an instance can be configured to allow specific built-in or external modules via `NODE_FUNCTION_ALLOW_BUILTIN` / `NODE_FUNCTION_ALLOW_EXTERNAL`). For anything needing a specific library or heavy compute, call a real service instead (6.4).

6.3 Python in the Code node

The Code node can also run **Python** (executed via Pyodide in the browser/engine sandbox). Handy if your team thinks in Python — but it's a sandboxed subset, not a full CPython with pip. For serious Python, expose it as an API and call it with the HTTP Request node.

6.4 The escape hatch pattern

The healthiest way to use n8n at scale: keep the workflow as orchestration, and push genuinely complex logic *out* — into a Code node for small stuff, or into a real service the workflow calls (HTTP Request → your API / a queue) for anything heavy, stateful, or library-dependent.

DIAGRAM

```
[Trigger] → [Fetch data] → [HTTP Request → your /process service] → [Notify]
                        the hard logic lives in code you can test
```

COMMON MISTAKE

Cramming a 200-line Code node full of business logic into a workflow and treating the canvas as your IDE. You lose version control, tests, and code review. If the Code node grows past "a transformation," move that logic into a proper module/service with tests and let n8n call it. The Code node is a scalpel, not a codebase.

WHEN NOT TO USE THE CODE NODE

Don't drop into code for something a built-in node already does — a Set for field mapping, a Filter for conditions, an HTTP Request for a call. Reaching for JavaScript first defeats the purpose (less glue) and makes the workflow harder to read. Try the nodes; use Code for what they genuinely can't express.

EXERCISE 6.1

Take a list of items and, in a Code node (all-items mode), deduplicate them by `email`, add a `receivedAt` timestamp, and sort by name — returning proper `{ json }` items. Then rewrite the *dedupe* step using only built-in nodes (Sort + a Filter/Aggregate approach) and decide which you'd keep. Note where code was clearer and where the nodes were.

Part 7 · Credentials & Secrets

Automations are worthless without access to your services, and dangerous if that access leaks. n8n has a **Credentials** system — a first-class, encrypted store for secrets, kept separate from the workflows that use them. Treat it with the same seriousness as your app's secret management.

7.1 Credentials are separate from workflows

You create a credential once (e.g. "Slack - workspace token"), and nodes *reference* it — they never contain the secret. This means:

- The secret is stored **encrypted** (with the instance's encryption key), not in the workflow JSON.
- Exporting or sharing a workflow doesn't leak the secret — only a reference goes with it.
- One credential is reused across many workflows; rotate it in one place.

DIAGRAM

```
[Slack node] —uses→ (Credential: "Slack - workspace") ← secret lives here, encrypted
[Another workflow's Slack node] —uses→ same credential
```

NODE.JS MENTAL MODEL

Credentials are your `.env` + a secrets manager, but scoped to n8n and encrypted at rest. A node referencing a credential is code reading `process.env.SLACK_TOKEN` — except the value is centralized, encrypted, and never serialized into the workflow. The workflow is safe to export; the credential is not part of it.

7.2 The encryption key is everything

n8n encrypts credentials with an **encryption key**. If you don't set one, n8n generates it and stores it in the data dir — fine until you move to a new container without that dir and suddenly *every credential is unreadable*.

YAML

```
environment:
  - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY}    # set this yourself, keep it safe
```

COMMON MISTAKE

Not setting `N8N_ENCRYPTION_KEY`, then migrating to a new host/volume and finding all credentials broken because the auto-generated key was left behind. Set the key explicitly from day one, store it in *your* secret manager, and back it up like a database password. Losing it means re-entering every credential by hand.

7.3 Auth types you'll use

Most credentials are one of: **API key / token** (a header or query value), **Basic auth** (user/pass), **OAuth2** (n8n runs the OAuth dance and stores the refresh token for you), or a **service-specific** credential (a database URL, a cloud key pair). For the generic HTTP Request node, you attach a "Generic Credential" (header auth, query auth, OAuth2) so even un-integrated APIs get proper secret handling.

7.4 External secret stores (Enterprise)

On Enterprise, n8n can pull secrets from an external manager (Vault, AWS/GCP secret managers) via `$secrets`, so nothing sensitive lives in n8n at all. On Community, the encrypted credential store *is* your vault — which is why the encryption key and its backups matter so much.

PRODUCTION TIP

Give each integration its **own least-privilege credential** — a Slack token scoped to one channel, a database user that can only touch the tables the workflow needs, an API key with minimal scopes. If a workflow (or the instance) is ever compromised, tightly-scoped credentials limit the blast radius — the same principle as service accounts in your backend.

EXERCISE 7.1

Create an API-key credential for a service and use it from both a dedicated node and a generic HTTP Request node. Export the workflow to JSON and confirm the secret is **not** in the export (only a credential reference). Then set `N8N_ENCRYPTION_KEY`, restart, and verify credentials still decrypt — the check that your migration story is sound.

Part 8 · Error Handling & Reliability

A workflow that only works on the happy path is a demo. Production automations hit timeouts, 500s, rate limits, and bad data. n8n gives you per-node retries, branch-on-error, a global error workflow, and (with queue mode) safe re-execution. This is where you turn a toy into infrastructure.

8.1 Per-node retries and timeouts

Most nodes have **"Retry On Fail"** with a retry count and wait — set it on any node that calls something flaky (an HTTP request, a third-party node). Combined with a request timeout, it's exponential-backoff-lite without code.

DIAGRAM

```
[HTTP Request]  Retry On Fail: 3  Wait Between: 2s  Timeout: 20s
→ transient 502s and slow responses recover on their own
```

NODE.JS MENTAL MODEL

This is the `p-retry / try-with-backoff` you'd wrap a `fetch` in, as a checkbox. Set it on every node that touches the network. As in code, only retry **idempotent** operations blindly — retrying a non-idempotent POST can double a charge or a message (see 8.4).

8.2 Continue-on-error and error branches

By default a node error stops the workflow. Two ways to handle it instead:

- **"Continue On Fail"** — the node emits the error as data and the workflow keeps going, so you can handle partial failures across items.
- **The error output** — some nodes expose a second output for failed items; wire it to a different path (log it, route it, alert).

DIAGRAM

```
[HTTP Request] — success → [Save result]
                  |
                  — error  → [Log + Slack alert]
```

8.3 The global Error Trigger workflow

Build **one** workflow that starts with an **Error Trigger** node, and set it as the error workflow for your others. Whenever any of them fails, n8n runs it with the error details — so you get one place that notifies you (Slack, email, a ticket) for every failure, instead of wiring alerts into each workflow.

DIAGRAM

```
[Any workflow fails] → [Error Trigger] → [Format] → [Slack #alerts]
```

PRODUCTION TIP

Create the Error Trigger workflow first, before you build anything real, and point every new workflow at it (Settings → Error Workflow). It's the n8n equivalent of a global `unhandledRejection` handler plus Sentry — one wire, and nothing fails silently again.

8.4 Idempotency — the reliability chapter within the chapter

Retries and re-runs mean a step can execute **more than once**. If that step sends an email, charges a card, or creates a record, "at least once" becomes "sometimes twice." Make side-effecting steps **idempotent**:

- Use an **idempotency key** the downstream API supports (Stripe, many others).
- **Check-before-write**: look up whether the record exists before creating it.
- **Dedupe on a natural key** (order id, event id) so a replay is a no-op.

COMMON MISTAKE

Turning on "Retry On Fail" for a node that sends a message or creates a payment, without idempotency. A transient timeout *after* the action succeeded — but before n8n saw the response — triggers a retry and does it twice. Retry freely for reads and idempotent writes; for everything else, add an idempotency key or a check-before-write first. (This is the same lesson as the Queues cookbook's idempotency chapter — it applies identically here.)

WHEN NOT TO RETRY AT ALL

Don't retry a request that failed for a **permanent** reason — a 400/401/404, a validation error, malformed input. Retrying a deterministic failure just wastes time and hammers the API. Retry transient faults (timeouts, 429, 5xx); route permanent ones to the error branch and stop.

EXERCISE 8.1

Build a workflow that calls a flaky endpoint: enable Retry On Fail (3×, 2s), add an error branch that Slacks you on final failure, and register a global Error Trigger workflow. Then add a create-record step and make it idempotent (check-before-write on a natural key); prove that running the workflow twice with the same input creates the record only once.

Part 9 · AI Workflows: LLMs, Agents & RAG

n8n has grown a first-class set of **AI nodes** (built on LangChain), so you can build LLM chains, tool-using agents, and retrieval pipelines visually. If you've read the RAG and LLM material, this is the same architecture you'd code — LLM calls, tools, embeddings, a vector store — assembled as nodes.

9.1 The building blocks

n8n node	What it is	Code equivalent
Chat/LLM model (OpenAI, Anthropic, Ollama, ...)	the model call	your SDK client
Basic LLM Chain	prompt → model → output	one <code>messages.create</code>
AI Agent	a model that can call tools in a loop	the agent loop (Part 27, Python book)
Tool nodes	functions the agent may call	your tool-calling handlers
Vector Store (Pinecone, Qdrant, PGVector, in-memory)	store/retrieve embeddings	your vector DB
Embeddings	text → vectors	the embeddings API
Memory	conversation history	your message store

NODE.JS MENTAL MODEL

An **AI Agent** node is the exact loop from the LLM chapter — model proposes a tool call, you run the tool, feed the result back, repeat until it answers — except n8n runs the loop and you attach **Tool** nodes as the callable functions. A Tool can be an HTTP Request, a sub-workflow, or a built-in. You're wiring the same components; n8n owns the while-loop.

9.2 A basic LLM chain

The simplest AI workflow: a trigger, a Basic LLM Chain node with a model attached and a prompt (using expressions to inject item data), and something to do with the output.

DIAGRAM

```

[Webhook: {question}] → [Basic LLM Chain]  prompt: "Answer: {{ $json.question }}"
                        | (model: your OpenAI/Anthropic credential)
                        ▼
                    [Respond to Webhook]
  
```

9.3 An agent with a tool

Attach a model and one or more tools to an **AI Agent** node. The classic first agent: a chat model with an HTTP Request tool it can call to fetch live data.

DIAGRAM



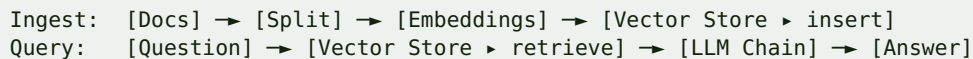
The agent decides *whether* to call the tool, does it, and folds the result into its answer — the mechanism behind "assistants." Cap tool iterations and treat tool inputs as untrusted, exactly as you would in code.

9.4 Retrieval (RAG) as a workflow

RAG splits into two workflows, matching the RAG cookbook's ingest/query split:

- **Ingest:** load documents → split into chunks → **Embeddings** → **Vector Store** (insert). Run it when your knowledge base changes.
- **Query:** question → embed → **Vector Store** (retrieve top-k) → stuff into the LLM prompt → answer.

DIAGRAM



PRODUCTION TIP

Use n8n to **prototype** an AI feature fast — wire an agent or a RAG flow in an afternoon and validate it's useful. If it becomes a core product path with real traffic, that's often the moment to reimplement the hot loop in code (your SDK + vector DB) and keep n8n for the surrounding orchestration. Same prototype-then-harden rule as everywhere else in this book.

9.5 MCP in n8n — tools over a standard protocol

n8n speaks **MCP (the Model Context Protocol)** in both directions, which lets it plug into the wider agent ecosystem:

- **MCP Client Tool node** — give an n8n **AI Agent** tools from an external MCP server (a filesystem, GitHub, database, or your own server) without building each integration by hand. The agent gets the server's tools automatically.
- **MCP Server Trigger** — expose an n8n workflow *as* an MCP server, so an outside AI client (Claude, an IDE, another agent) can call your workflow as a tool.

DIAGRAM

```

n8n AI Agent —MCP Client→ external MCP server (its tools)
external AI client —MCP→ n8n MCP Server Trigger (your workflow as a tool)

```

MCP is a protocol, so this is unrelated to JavaScript vs Python — it works the same regardless of what your Code nodes use.

NODE.JS MENTAL MODEL

MCP here is the same idea as the AI Agents book's MCP chapter, wired visually: the **Client** node is your agent using someone else's tool server; the **Server Trigger** turns your workflow into a reusable tool others can call. It's n8n joining the standard tool bus instead of every integration being bespoke. The security caveats carry over — tool arguments and any data a server returns are untrusted; scope access and guard destructive actions.

COMMON MISTAKE

Shipping an AI workflow with no cost or rate controls. Every model and embeddings node is a metered call; an agent can make several per run, and a busy trigger multiplies that. Add batching, cap agent iterations, cache where you can, and log token usage — the LLM cookbook's cost patterns apply unchanged. An unwatched agent loop is a surprise bill.

WHEN NOT TO BUILD THE AI FLOW IN N8N

If the feature needs fine control over streaming, custom retrieval logic, evaluation harnesses, or tight latency, the visual nodes will fight you — code it with the SDK. n8n's AI nodes are best for straightforward chains, internal agents, and prototypes, not for a bespoke, performance-critical AI product.

EXERCISE 9.1

Build two workflows: (1) an AI Agent with a single HTTP Request tool it can call to answer a question that needs live data; (2) a mini-RAG pair — ingest a few documents into an in-memory or PGVector store, then answer a question by retrieving and prompting. Add token-usage logging and an iteration cap on the agent.

Part 10 · Self-Hosting: Persistence, Queue Mode & Security

This is the chapter that decides whether your n8n is a demo or infrastructure. The default single-container, SQLite setup is fine for learning; a real deployment needs a proper database, workers, and a locked-down instance. Everything here is Community-edition capable.

10.1 Move the database to Postgres

n8n stores its workflows, credentials, and execution history in a database. SQLite (the default) doesn't handle concurrency or scale; switch to **Postgres**:

YAML

```
services:
  postgres:
    image: postgres:16
    environment:
      - POSTGRES_USER=n8n
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=n8n
    volumes:
      - pg_data:/var/lib/postgresql/data

  n8n:
    image: docker.n8n.io/n8nio/n8n
    depends_on: [postgres]
    ports: ["5678:5678"]
    environment:
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=postgres
      - DB_POSTGRESDB_DATABASE=n8n
      - DB_POSTGRESDB_USER=n8n
      - DB_POSTGRESDB_PASSWORD=${POSTGRES_PASSWORD}
      - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY}
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  pg_data:
  n8n_data:
```

NODE.JS MENTAL MODEL

Same instinct as any stateful Node service: don't keep production data in a file inside the container. Postgres + a volume is the baseline, and `N8N_ENCRYPTION_KEY` set explicitly (Part 7) means your credentials survive a redeploy. This compose is the "it survives `docker compose down`" version.

10.2 Queue mode — the part that makes it scale

By default n8n runs everything in one process (**main mode**). Under load, or for long-running workflows, switch to **queue mode**: the main process just accepts triggers and pushes executions onto **Redis**, and separate **worker** processes pull and run them.

DIAGRAM

```

[main] —enqueue→ (Redis) → [worker] [worker] [worker]
triggers/UI      broker    run the executions in parallel

```

YAML

```

n8n:                                     # main: UI + triggers
  environment:
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis
    # ...same DB + encryption env as above

n8n-worker:                             # one or more workers
  image: docker.n8n.io/n8nio/n8n
  command: worker
  environment:
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis
    # ...same DB + encryption env

redis:
  image: redis:7

```

NODE.JS MENTAL MODEL

This is **exactly** BullMQ, because it is BullMQ under the hood: main = producer, Redis = broker, `command: worker` = your worker process. Scaling out is "run more worker containers" — the same horizontal-scaling story as the Queues cookbook. If that book made sense, queue mode is free.

10.3 Security essentials

A self-hosted n8n reaches into your services and holds your secrets. Lock it down:

- **Encryption key** — `N8N_ENCRYPTION_KEY` set and backed up (Part 7).

- **Auth** — the owner account protects the editor; never expose an unauthenticated instance to the internet.
- **HTTPS** — put it behind a reverse proxy (Caddy/Nginx/Traefik) for TLS; set `N8N_HOST`, `N8N_PROTOCOL=https`, and the correct `WEBHOOK_URL` so webhook URLs are right.
- **Network** — keep Postgres/Redis on an internal network, not published ports.
- **Updates** — pin and update the image; watch releases for security fixes.

COMMON MISTAKE

Exposing n8n directly on a public IP with `N8N_SECURE_COOKIE=false` (a dev setting) and no proxy. That's an internet-facing box holding all your API credentials with cookies sent in the clear. For anything public: TLS via a proxy, secure cookies on, the owner account enabled, and datastores unpublished.

PRODUCTION TIP

Set `WEBHOOK_URL` to your public HTTPS address explicitly. Behind a proxy, n8n otherwise generates webhook URLs from the internal host and the URLs it shows you won't be reachable — the classic "my production webhook 404s" self-host bug.

EXERCISE 10.1

Stand up the Postgres compose, confirm workflows persist across a full `down / up`. Then enable queue mode with Redis and one worker; run a slow workflow (add a Wait node) and confirm it executes on the **worker** (check its logs), not the main process. Add a second worker and watch load spread.

Part 11 · Production Practices

You know how to build and self-host workflows. This chapter is about running them like an engineer: versioning, environments, testing, monitoring, and a clear-eyed list of when to walk away from n8n entirely.

11.1 Version control your workflows

Workflows are **JSON** — so treat them like code. You can export a workflow to JSON and commit it, and n8n's API/CLI lets you pull and push workflows so a Git repo is the source of truth.

SHELL

```
# export all workflows to files (via the n8n CLI inside the container)
docker exec n8n n8n export:workflow --all --separate --output=/data/workflows
# ...commit that directory; import on deploy with n8n import:workflow
```

NODE.JS MENTAL MODEL

A workflow is a JSON artifact like a config file. The discipline is the same as infra-as-code: edit in the UI, export, commit, review the diff, deploy by importing. Don't let the only copy of a critical workflow live in one instance's database — that's an un-backed-up production server.

11.2 Environments: dev vs prod

Keep a **separate n8n instance** (or at least separate credentials and a staging database) for building vs running. Credentials with the *same name* but different values per environment let the same workflow point at test vs live services without edits. (First-class "environments" and Git integration are Enterprise features; on Community you approximate with separate instances + naming conventions.)

11.3 Testing

n8n has no unit-test framework like your app does, so lean on what it gives you:

- **Pin data** on trigger nodes to a representative sample and re-run to check behavior after changes.
- **Push complex logic into a Code node or a service** that you *can* unit-test in your normal toolchain, and keep the workflow thin.
- Keep a **staging workflow** that exercises the path end-to-end against test services before you activate the production one.

PRODUCTION TIP

The most testable n8n architecture keeps workflows as orchestration and puts real logic in services with their own tests (Part 6's escape-hatch pattern). Then "testing the workflow" is just checking the wiring — the hard parts are covered by ordinary unit tests in a repo you control.

11.4 Monitoring

- **Executions list** — your first stop; filter to failures.
- **The global Error Trigger workflow** (Part 8) — active alerting on any failure.
- **Health/metrics** — n8n exposes a health endpoint and (optionally) Prometheus metrics; scrape them like any service.
- **Log retention** — prune execution data (Part 3) so the database stays healthy.

11.5 The honest "when NOT to use n8n" list

WHEN NOT TO USE N8N

Walk away — write code — when the work is **latency-critical** (n8n's per-node overhead is real), **very high throughput** (a lean worker beats it), **complex branching logic** (clearer in a codebase than a sprawling canvas), a **core product feature** needing strict SLAs and deep test coverage, or **trivial and stable** (a cron one-liner doesn't need an engine). n8n's sweet spot is integration glue that changes often and benefits from being visible and editable — internal automations, ops, notifications, and prototypes.

COMMON MISTAKE

Letting n8n sprawl into unmanaged shadow infrastructure — dozens of business-critical workflows, no version control, no owner, secrets only in one box. That's the automation equivalent of an undocumented server under someone's desk. If n8n is doing important work, give it the same rigor as the rest of your stack: Git-backed workflows, backups (DB + encryption key), monitoring, and an owner.

EXERCISE 11.1

Take one workflow to "production-ready": export it to JSON and commit it, point it at a global Error Trigger workflow, set same-named dev/prod credentials, pin sample data on its trigger for repeatable testing, and write a one-paragraph runbook (what it does, what breaks it, who owns it). This is the difference between a clever workflow and a maintainable one.

Part 12 · Capstone & Cheat Sheet

12.1 Capstone — build one real automation end to end

Pick one automation from your Exercise 1.1 list and build it properly, using everything in the book. A good reference capstone:

"New signup → enrich → store → notify," done right.

1. **Trigger** — a Webhook node receives `{ email, name }` from your app (activated production URL, Part 3).
2. **Enrich** — an HTTP Request node calls an enrichment API using a scoped credential (Parts 5, 7), driven by `{{ $json.email }}`.
3. **Transform** — a Set node builds a clean record; a Code node handles anything the built-in nodes can't (Parts 4, 6).
4. **Store** — a Postgres node inserts the record, made **idempotent** on the email so replays don't duplicate (Part 8).
5. **Notify** — a Slack node posts to `#signups`.
6. **Reliability** — Retry On Fail on the network nodes, an error branch, and a global Error Trigger workflow watching it (Part 8).
7. **Ship it** — run on Postgres + queue mode, secrets via credentials with a set encryption key, workflow exported to Git, pointed at dev vs prod credentials (Parts 10, 11).

If you can build that, you can build most of what n8n is good for — and you know where its edges are.

12.2 Node.js → n8n mental-model map

You'd write...	In n8n it's...
<code>app.post("/hook")</code>	Webhook trigger
<code>node-cron</code> / cron entry	Schedule trigger
<code>fetch</code> / <code>axios</code>	HTTP Request node
<code>arr.map()</code> over data	a node running once per item
<code>.filter()</code>	Filter node
flatten an array to many	Split Out node
<code>.reduce()</code> / combine	Aggregate node
<code>if/else</code> , <code>switch</code>	IF / Switch nodes
a helper function	Code node
calling another module	Execute Sub-workflow node

You'd write...	In n8n it's...
<code>process.env</code> / secrets manager	Credentials
<code>try/catch</code> + <code>retry</code>	Retry On Fail + error branch
global error handler + Sentry	Error Trigger workflow
BullMQ (producer/broker/worker)	queue mode (main/Redis/worker)
the agent loop	AI Agent node + Tool nodes
your vector DB	Vector Store node

12.3 Expressions cheat sheet

DIAGRAM

<code>{{ \$json.field }}</code>	current item's field
<code>{{ \$json.a.b[0] }}</code>	nested access
<code>{{ \$json.name.toUpperCase() }}</code>	real JS methods
<code>{{ \$node["Get User"].json.id }}</code>	a named earlier node's output
<code>{{ \$('Get User').item.json.id }}</code>	same, newer syntax
<code>{{ \$now.toISO() }}</code>	current time (Luxon)
<code>{{ \$now.minus({days:7}) }}</code>	date math
<code>{{ \$env.MY_VAR }}</code>	an env var (if enabled)
<code>{{ \$itemIndex }}</code>	index of the current item
<code>{{ {{expr}} ? 'a' : 'b' }}</code>	ternaries, template logic

12.4 Self-hosting env cheat sheet

DIAGRAM

<code>N8N_ENCRYPTION_KEY</code>	encrypts credentials – set + back up (critical)
<code>DB_TYPE=postgresdb</code>	use Postgres, not SQLite
<code>DB_POSTGRESDB_*</code>	host / database / user / password
<code>EXECUTIONS_MODE=queue</code>	turn on queue mode (workers)
<code>QUEUE_BULL_REDIS_HOST</code>	Redis for queue mode
<code>N8N_HOST / N8N_PROTOCOL</code>	public host + https (behind a proxy)
<code>WEBHOOK_URL</code>	public base URL for webhook links
<code>GENERIC_TIMEZONE / TZ</code>	correct times for Schedule triggers
<code>EXECUTIONS_DATA_MAX_AGE</code>	prune old execution history

12.5 The five rules

1. **Everything is a list of items;** nodes run once per item.
2. **Try the built-in nodes first;** drop into the Code node only for what they can't express.
3. **Secrets live in Credentials,** with the encryption key set and backed up.
4. **Nothing fails silently** — Retry On Fail, error branches, and a global Error Trigger workflow.
5. **Self-host it for real** — Postgres, queue mode, TLS — or it's a demo.

YOU'RE READY

You can now build automations visually, escape into JavaScript when you need to, secure the secrets, handle failure, add AI, and self-host it at scale — and you know the cases where writing the service by hand is the right call instead. That last judgment is what separates an engineer using n8n from someone fighting it. Go automate the boring parts.